

Design Patterns

These deal with how the user interface (UI), business logic, and data are separated and interact, usually within a single application or component.

Key Differences

Feature	System Architecture Patterns	UI Architecture Patterns
Scope	Entire system/application	Mainly the UI layer
Focus	How components/services are structured and deployed	How UI logic is separated and organized
Granularity	High-level, coarse-grained	Low-level, fine-grained
Examples	Microservices, Layered, Microkernel	MVC, MVP, MVVM
Purpose	Scalability, maintainability, deployability	Separation of concerns, testability, maintainability of UI

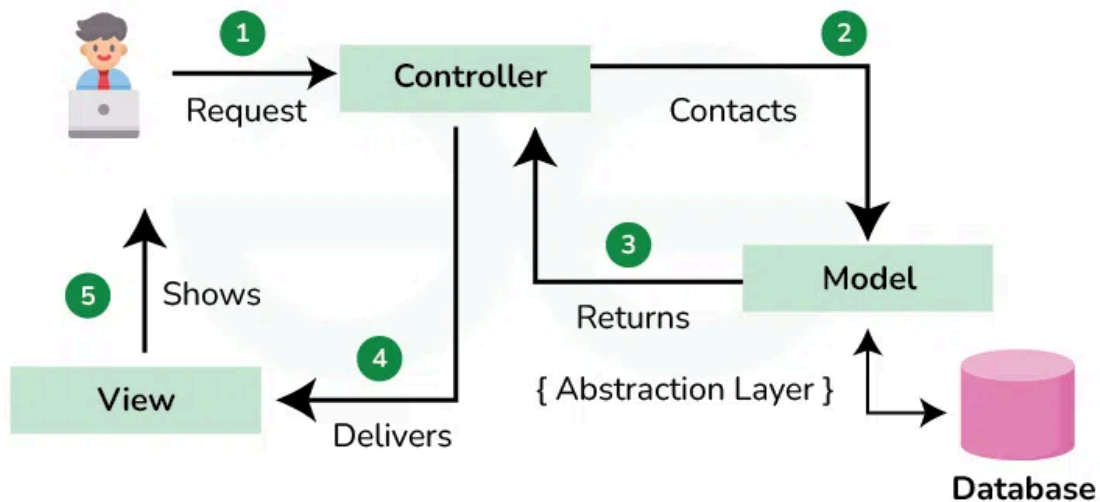
MVC (Model-View-Controller)

Model: Handles data and business logic

View: Handles the UI

Controller: Mediates input, updates model and view

Used in: Web frameworks (e.g., Ruby on Rails, Django, ASP.NET MVC)

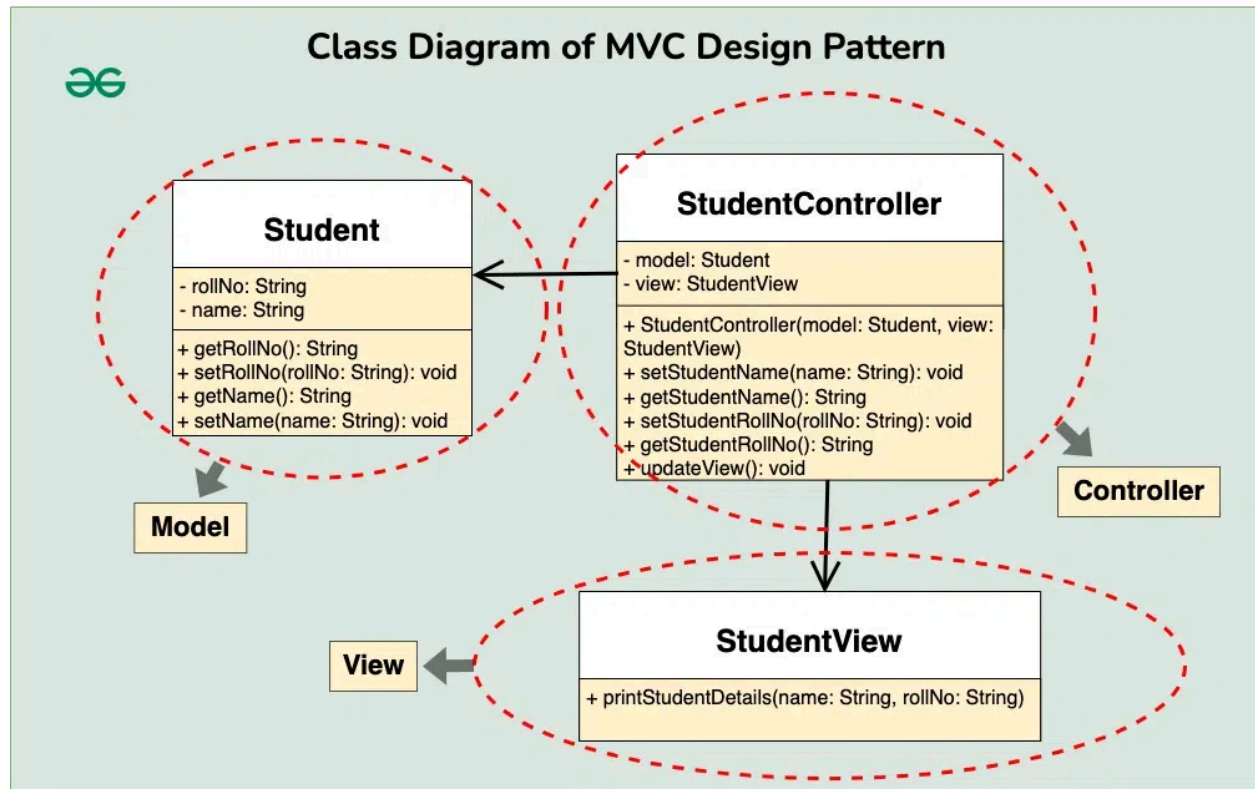


Communication between the Components

This below communication flow ensures that each component is responsible for a specific aspect of the application's functionality, leading to a more maintainable and scalable architecture

- **User Interaction with View:** The user interacts with the View, such as clicking a button or entering text into a form.
- **View Receives User Input:** The View receives the user input and forwards it to the Controller.
- **Controller Processes User Input:** The Controller receives the user input from the View. It interprets the input, performs any necessary operations (such as updating the Model), and decides how to respond.
- **Controller Updates Model:** The Controller updates the Model based on the user input or application logic.
- **Model Notifies View of Changes:** If the Model changes, it notifies the View.
- **View Requests Data from Model:** The View requests data from the Model to update its display.
- **Controller Updates View:** The Controller updates the View based on the changes in the Model or in response to user input.
- **View Renders Updated UI:** The View renders the updated UI based on the changes made by the Controller.

Example:



Code implementation:

Model:

```
class Student {  
    private String rollNo;  
    private String name;  
  
    public String getRollNo() {  
        return rollNo;  
    }  
  
    public void setRollNo(String rollNo) {  
        this.rollNo = rollNo;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

View :

```
class StudentView {  
    public void printStudentDetails(String studentName, String studentRollNo) {  
        System.out.println("Student:");  
        System.out.println("Name: " + studentName);  
        System.out.println("Roll No: " + studentRollNo);  
    }  
}
```

Controller:

```
class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name) {
        model.setName(name);
    }

    public String getStudentName() {
        return model.getName();
    }

    public void setStudentRollNo(String rollNo) {
        model.setRollNo(rollNo);
    }

    public String getStudentRollNo() {
        return model.getRollNo();
    }

    public void updateView() {
        view.printStudentDetails(model.getName(), model.getRollNo());
    }
}
```



```

public class MVCPattern {
    public static void main(String[] args) {
        Student model = retrieveStudentFromDatabase();

        StudentView view = new StudentView();

        StudentController controller = new StudentController(model, view);

        controller.updateView();

        controller.setStudentName("Vikram Sharma");

        controller.updateView();
    }

    private static Student retrieveStudentFromDatabase() {
        Student student = new Student();
        student.setName("Lokesh Sharma");
        student.setRollNo("15UCS157");
        return student;
    }
}

```

MVP

In the initial stages of **Android development**, learners do write codes in such a manner that eventually creates a **MainActivity class which contains all the implementation logic(real-world business logic) of the application**. This approach of app development leads to Android activity gets closely coupled to both UI and the application data processing mechanism. Further, it causes difficulties in the maintenance and scaling of such mobile applications. To avoid such problems in maintainability, readability, scalability, and refactoring of applications, developers prefer to define well-separated layers of code. By applying software architecture patterns, one can organize the code of the application to separate the concerns. MVP (Model - View - Presenter) architecture is one of the most popular architecture patterns and is valid in organizing the project.

MVP (Model - View - Presenter) comes into the picture as an alternative to the traditional MVC (Model - View - Controller) architecture pattern. Using MVC as the software architecture, developers end up with the following difficulties:

- Most of the core business logic resides in Controller. During the lifetime of an application, this file grows bigger and it becomes difficult to maintain the code.
- Because of tightly-coupled UI and data access mechanisms, both Controller and View layer falls in the same activity or fragment. This cause problem in making changes in the features of the application.
- It becomes hard to carry out Unit testing of the different layer as most of the part which are under testing need Android SDK components.

MVP pattern overcomes these challenges of MVC and provides an easy way to structure the project codes. The reason why MVP is widely accepted is that it provides modularity, testability, and a more clean and maintainable codebase. It is composed of the following three components:

- Model: Layer for storing data. It is responsible for handling the domain logic(real-world business rules) and communication with the database and network layers.
- View: UI(User Interface) layer. It provides the visualization of the data and keep a track of the user's action in order to notify the Presenter.
- Presenter: Fetch the data from the model and applies the UI logic to decide what to display. It manages the state of the View and takes actions according to the user's input notification from the View.